

Supervised vs. unsupervised learning

COMPLETE LEARNING GUIDE

What You'll Learn

Almost every machine learning technique falls into one of two broad camps, and knowing which camp a problem belongs to is the very first decision in any ML project — it determines what kind of data you need, which algorithms are even applicable, and how you'll evaluate success. Supervised learning learns from labeled examples (input paired with the correct output) to predict labels for new, unseen inputs; unsupervised learning finds structure in unlabeled data with no 'correct answer' provided at all. This submodule covers the core distinction, the two main supervised sub-tasks (classification and regression), the main unsupervised task (clustering), and grounds each in concrete, realistic use cases so the distinction isn't just theoretical — by the end you should be able to look at a new business problem and immediately identify which category it falls into.

1. The Core Distinction: Does Your Data Have Labels?

Supervised learning requires a labeled dataset — for every input example, you already know the correct output (e.g. thousands of emails, each already marked spam or not-spam). The algorithm's job is to learn the mapping from input features to output label well enough to generalize to new, unlabeled examples. Unsupervised learning works with data that has no labels at all — there's no 'correct answer' given for any example; instead, the algorithm looks for inherent structure, patterns, or groupings in the data itself. The practical consequence is significant: supervised learning requires you to have (or create) labeled training data, which is often the most expensive and time-consuming part of a real project, while unsupervised learning can be applied directly to raw, unlabeled data you already have.

2. Classification vs. Regression: The Two Faces of Supervised Learning

Within supervised learning, the nature of the label determines which sub-task you're solving. Classification predicts a discrete category — spam/not-spam, which of five product categories a listing belongs to, whether a transaction is fraudulent. Regression predicts a continuous numeric value — a house's sale price, tomorrow's temperature, how many units will sell next month. The distinction matters because it determines which algorithms apply (some, like linear regression, are regression-only; others, like logistic regression despite its name, are classification-only) and which evaluation metrics make sense (accuracy/precision/recall for classification; mean squared error/ R^2 for regression).

- Classification: predicting a label from a fixed, discrete set of categories (binary — two classes — or multiclass — three or more).
- Regression: predicting an unbounded (or bounded-but-continuous) numeric quantity.
- The same underlying algorithm family often has both flavors — e.g. decision trees and random forests can do either classification or regression depending on configuration.

3. Clustering: The Main Unsupervised Task

Clustering groups similar data points together without being told in advance what the groups should be or how many there should be. K-means clustering (the most common starting point) partitions data into k groups by iteratively assigning each point to its nearest cluster center and then recomputing each center as the average of its assigned points, repeating until the assignments stabilize. Hierarchical clustering instead builds a tree of nested groupings (a dendrogram) by successively merging or splitting clusters, which is useful when you don't want to commit to a fixed number of clusters up front. Unlike classification, there's no 'correct' cluster assignment to check your answer against — success is judged by whether the resulting groups are useful and interpretable for the problem at hand.

- K-means requires you to choose k (the number of clusters) in advance — often chosen using the 'elbow method' or domain knowledge.
- Clusters found by an algorithm still need a human to interpret and name them ('these customers all seem to be high-frequency, low-value buyers') — the algorithm doesn't label the groups for you.
- Clustering quality metrics (like silhouette score) can suggest good structure exists, but usefulness ultimately depends on whether the groups are actionable for your specific goal.

4. Concrete Use Cases Across Both Categories

Grounding the categories in realistic scenarios makes the distinction stick. Classification: email spam detection, medical diagnosis from test results (disease present/absent), credit approval (approve/deny). Regression: predicting house prices from square footage and location, forecasting next quarter's revenue, estimating a delivery time. Clustering: grouping customers into segments for targeted marketing without predefined segment definitions, grouping news articles by topic, detecting anomalous network traffic patterns that don't fit any known cluster. Recognizing which bucket a real business question falls into — 'do I have labels?' then 'is the label discrete or continuous?' — is the practical skill this section is building.

- 'Will this customer churn?' !' classification (label: churn / no churn, known historically for past customers).
- 'How much will this customer spend next month?' !' regression (label: a dollar amount).
- 'What natural customer segments exist in our data?' !' clustering (no predefined segment labels exist at all).

5. A Quick Look at the Code Difference

The scikit-learn API reflects the conceptual split directly: supervised estimators' `.fit()` method takes both X (features) and y (labels), while unsupervised estimators' `.fit()` takes only X. This single API difference is a useful quick check for which category any given scikit-learn algorithm belongs to.

PYTHON

```
from sklearn.linear_model import LogisticRegression # supervised:
# classification
from sklearn.linear_model import LinearRegression # supervised: regression
from sklearn.cluster import KMeans # unsupervised:
clustering

# Supervised: fit() needs both features (X) and known labels (y)
clf = LogisticRegression().fit(X_train, y_train)
predictions = clf.predict(X_test)

# Unsupervised: fit() needs only features (X) — there is no y
kmeans = KMeans(n_clusters=3, n_init=10).fit(X)
cluster_labels = kmeans.labels_
```

6. Beyond the Two Categories: A Brief Mention of What's Next

Supervised and unsupervised learning cover the large majority of practical ML use cases and are the two categories this course focuses on, but it's worth knowing two related terms exist: semi-supervised learning (a small amount of labeled data combined with a much larger amount of unlabeled data) and reinforcement learning (an agent learns by taking actions in an environment and receiving rewards or penalties, rather than learning from a fixed labeled dataset at all — the paradigm behind game-playing AI and robotics control). Neither is covered in depth in this course, but recognizing the terms helps you correctly categorize problems you encounter that don't neatly fit supervised or unsupervised.

Common Pitfalls

-
- Trying to force a genuinely unlabeled problem into a supervised approach by inventing arbitrary labels, rather than reaching for clustering.
 - Using classification evaluation metrics (accuracy, precision, recall) on a regression problem, or vice versa — the metrics aren't interchangeable.
 - Assuming k-means will find 'the' correct number of clusters on its own — k must be chosen, and different choices produce meaningfully different groupings.
 - Confusing logistic regression (a classification algorithm, despite the name) with linear regression (a regression algorithm) based on the name alone.
 - Expecting clustering output to come with meaningful group names or explanations — interpreting and naming clusters is a manual, domain-expertise step.
 - Treating a naturally ordinal label (like a 1-5 star rating) as either purely categorical or purely continuous without considering which framing better serves the actual business question.

Key Takeaways

- The first question for any ML problem is 'do I have labeled data?' — that alone determines supervised vs. unsupervised.
- Within supervised learning, a discrete label means classification; a continuous label means regression.
- Clustering finds structure in unlabeled data but never tells you what the groups mean — that interpretation is a human step.
- scikit-learn's API mirrors the concept directly: supervised `.fit(X, y)` vs. unsupervised `.fit(X)`.
- Correctly categorizing a real business question (labels? discrete or continuous?) is the practical skill, more than memorizing algorithm names.

Learn More

scikit-learn — Supervised Learning User Guide

https://scikit-learn.org/stable/supervised_learning.html

scikit-learn — Unsupervised Learning (Clustering) User Guide

<https://scikit-learn.org/stable/modules/clustering.html>

Google Developers — Machine Learning Crash Course: Framing

<https://developers.google.com/machine-learning/crash-course>